

Question 1: Water Jug Problem

Aim

To solve the Water Jug Problem using Artificial Intelligence and obtain exactly 2 gallons in the 4-gallon jug.

Problem Statement

You are given two jugs:

Jug A = 4 gallons

Jug B = 3 gallons

Initially, both jugs are empty. You can:

Fill a jug completely.

Empty a jug.

Pour water from one jug to another until one jug becomes empty or the other becomes full.

Find a sequence of steps to obtain exactly 2 gallons in the 4-gallon jug.

State Space Representation

A state is represented as (A, B) where:

A = amount of water in the 4-gallon jug.

B = amount of water in the 3-gallon jug.

Algorithm

1. Start with both jugs empty (0,0).

2. Fill the 3-gallon jug.

3. Pour water into the 4-gallon jug.
4. Repeat filling, emptying, and pouring until the 4-gallon jug contains exactly 2 gallons.
5. Stop when the goal state is reached.

Step-by-Step Solution

Step	Action	State (4-gallon, 3-gallon)
1	Start	(0,0)
2	Fill 3-gallon jug	(0,3)
3	Pour into 4-gallon jug	(3,0)
4	Fill 3-gallon jug again	(3,3)
5	Pour into 4-gallon until full	(4,2)
6	Empty 4-gallon jug	(0,2)
7	Pour remaining 2 gallons into 4-gallon jug	(2,0) ✓ Goal

Pseudocode

```
Start
A = 0, B = 0
Fill B
Pour B → A
Fill B
Pour B → A until A is full
Empty A
Pour remaining water from B → A
Stop when A = 2
```

Python Code

```
# Water Jug Problem (4-gallon and 3-gallon)

steps = [
    (0, 0),
    (0, 3),
    (3, 0),
    (3, 3),
```

```

(4, 2),
(0, 2),
(2, 0)
]

print("Water Jug Problem Solution:")
for i, state in enumerate(steps):
    print(f"Step {i}: 4-gallon = {state[0]}, 3-gallon = {state[1]}")

print("\nGoal Achieved: 2 gallons in the 4-gallon jug.")

```

Expected Output

```

Water Jug Problem Solution:
Step 0: 4-gallon = 0, 3-gallon = 0
Step 1: 4-gallon = 0, 3-gallon = 3
Step 2: 4-gallon = 3, 3-gallon = 0
Step 3: 4-gallon = 3, 3-gallon = 3
Step 4: 4-gallon = 4, 3-gallon = 2
Step 5: 4-gallon = 0, 3-gallon = 2
Step 6: 4-gallon = 2, 3-gallon = 0

```

Goal Achieved: 2 gallons in the 4-gallon jug.

Result

The Water Jug Problem is solved successfully. The goal state (2,0) is reached, meaning the 4-gallon jug contains exactly 2 gallons.

Question 2: Mars Rover Agent

Aim

To design a goal-based intelligent agent that enables a Mars Rover to navigate the Martian surface, avoid obstacles, and reach a target location.

Problem Statement

A Mars Rover explores the surface of Mars. The rover must move from its starting position to a target location while avoiding rocks and other obstacles. The rover uses sensors to detect obstacles and chooses the best path to safely reach the goal.

Solution

The Mars Rover acts as a goal-based agent. It continuously senses its surroundings, decides the next safe move, and navigates toward the destination. If an obstacle is detected, it changes direction and continues moving until it reaches the target.

Algorithm

1. Start the rover at the initial position.
2. Sense the environment for obstacles.
3. If the path is clear, move forward.
4. If an obstacle is detected, choose an alternate safe path.
5. Repeat until the destination is reached.
6. Stop.

Pseudocode

```
START
Initialize rover position
WHILE destination not reached
    Sense surroundings
    IF obstacle detected
        Change direction
    ELSE
        Move forward
END WHILE
STOP
```

Python Code

```
print("Mars Rover Agent")

destination = False
```

```
while not destination:
    obstacle = False

    if obstacle:
        print("Obstacle detected! Changing direction...")
    else:
        print("Moving towards destination...")
        destination = True

print("Destination reached successfully.")
```

Expected Output

Mars Rover Agent
Moving towards destination...
Destination reached successfully.

Result

The Mars Rover Agent successfully reaches the destination by sensing obstacles and selecting a safe path.

i. What are the percepts for this agent?

Answer:

Percepts are the information the Mars Rover receives from its sensors, such as:

- Obstacles (rocks, hills)
- Terrain condition
- Position/location
- Weather
- Battery level

ii. Characterize the operating environment.

Answer:

The Mars Rover operates in a **partially observable, dynamic, sequential, and uncertain** environment on the surface of Mars.

iii. What actions can the agent take?

Answer:

The Mars Rover can:

- Move forward
- Turn left
- Turn right
- Stop
- Avoid obstacles
- Collect rock/soil samples

- Capture images
- Send data to Earth

iv. How can one evaluate the performance of the agent?

Answer:

The performance is evaluated based on:

- Reaching the target safely
- Avoiding obstacles
- Collecting correct samples
- Using less time and energy
- Successfully sending data to Earth

v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer:

A **Goal-Based Agent** is the most suitable because it plans actions to achieve a specific goal, such as reaching a destination and collecting samples while avoiding obstacles. It can make decisions based on the current situation to accomplish its mission effectively.

Question 3: 8-Queens Problem

Aim

To solve the 8-Queens Problem using Artificial Intelligence by placing eight queens on a chessboard so that no two queens attack each other.

Problem Statement

Place 8 queens on an 8×8 chessboard such that no two queens share the same row, column, or diagonal.

Solution

The 8-Queens problem is solved using the Backtracking Algorithm. Queens are placed one row at a time. If a queen can be placed safely, the algorithm moves to the next row. If a conflict occurs, it backtracks and tries another position until all eight queens are placed.

Algorithm

1. Start with an empty 8×8 chessboard.
2. Place a queen in the first row at a safe position.
3. Move to the next row and place another queen safely.

4. If no safe position exists, backtrack to the previous row.

5. Repeat until all 8 queens are placed.

6. Display the solution.

Pseudocode

START

Place queen in first row

FOR each row

 IF safe position exists

 Place queen

 ELSE

 Backtrack

Display solution

STOP

Python Code

N = 8

board = [[0] * N for _ in range(N)]

def is_safe(board, row, col):

 for i in range(col):

 if board[row][i]:

 return False

 i, j = row, col

 while i >= 0 and j >= 0:

 if board[i][j]:

 return False

 i -= 1

 j -= 1

 i, j = row, col

 while i < N and j >= 0:

```

        if board[i][j]:
            return False
        i += 1
        j -= 1

    return True

def solve(board, col):
    if col >= N:
        return True

    for i in range(N):
        if is_safe(board, i, col):
            board[i][col] = 1

            if solve(board, col + 1):
                return True

            board[i][col] = 0

    return False

if solve(board, 0):
    print("8-Queens Solution:")
    for row in board:
        print(row)
else:
    print("No Solution Exists")

```

Expected Output

```

8-Queens Solution:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]

```

Result

The 8-Queens Problem was successfully solved using the Backtracking Algorithm, placing all eight queens on the chessboard without any two queens attacking each other.

Question 4: OLA Cab Problem-Solving Agent

Aim

To design a problem-solving agent that helps an OLA cab choose the best route and reach the passenger and destination efficiently.

Problem Statement

An OLA Cab receives a ride request from a passenger. The cab agent must determine the shortest and safest route, avoid traffic when possible, pick up the passenger, and reach the destination with minimum travel time.

Solution

The OLA Cab acts as a problem-solving agent. It collects information such as the passenger's location, destination, traffic conditions, and available routes. The agent evaluates the possible paths and selects the most efficient one to minimize travel time and distance.

Algorithm

1. Start the OLA Cab agent.
2. Receive the passenger's pickup and destination locations.
3. Find all possible routes.
4. Select the shortest or fastest route.
5. Navigate to the pickup location.
6. Pick up the passenger.
7. Travel to the destination.

8. End the trip.

Pseudocode

```
START
Receive pickup and destination
Find available routes
Select the shortest route
Go to pickup location
Pick up passenger
Move to destination
Trip completed
STOP
```

Python Code

```
# Problem-Solving Agent

def problem_solving_agent(goal):

    if goal == "College":
        return "Go to College"
    elif goal == "Hospital":
        return "Go to Hospital"
    elif goal == "Home":
        return "Go to Home"
    elif goal == "Office":
        return "Go to Office"
    else:
        return "Goal Not Found"

goal = input("Enter Goal: ")

result = problem_solving_agent(goal)

print("Goal:", goal)
print("Action:", result)
```

Expected Output

Goal: College
Action: Go to College

Result

The OLA Cab Problem-Solving Agent successfully selected an efficient route, picked up the passenger, and reached the destination, demonstrating the working of a problem-solving agent in Artificial Intelligence

.

Question 5: Uniform Cost Search (UCS) on the Given Graph

Aim

To find the **least-cost path** from the start node **S** to the goal node **G** using the **Uniform Cost Search (UCS)** algorithm.

Problem Statement

A weighted graph is given with nodes and edge costs. The objective is to use the **Uniform Cost Search (UCS)** algorithm to find the **minimum-cost path** from the start node **S** to the goal node **G**

.

Solution

Uniform Cost Search expands the node with the **lowest cumulative cost** first. It continues exploring the graph until it reaches the goal node with the minimum total cost.

Algorithm

1. Start from node **S**.
2. Insert the start node into a priority queue with cost **0**.
3. Remove the node with the smallest cost.
4. If it is the goal node, stop.
5. Otherwise, add all unvisited neighboring nodes with their cumulative costs.
6. Repeat until the goal is found.

Pseudocode

START

Insert start node into priority queue

WHILE queue is not empty

 Remove node with minimum cost

 IF node is goal

 Display path and cost

```
        STOP
    Add neighboring nodes with updated costs
END WHILE
STOP
```

Python Code

```
import heapq

graph = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}

queue = [(0, 'S')]
visited = set()

while queue:
    cost, node = heapq.heappop(queue)

    if node in visited:
        continue

    visited.add(node)

    if node == 'G':
        print("Goal Reached")
        print("Minimum Cost:", cost)
        break

    for neighbor, weight in graph[node]:
        if neighbor not in visited:
            heapq.heappush(queue, (cost + weight, neighbor))
```

Sample Output

```
Goal Reached
Minimum Cost: 4
```

Result

The **Uniform Cost Search (UCS)** algorithm successfully found the **minimum-cost path** from **S** to **G**. The minimum cost is **4**.